

Fraud Detection Report

From Classical Data Analysis to Neural Networks

Python for Business Economics and Finance — Mid-Term Assignment · March 2026

12,000	24	2.13%
Transactions	Variables	Fraud Rate

1. Introduction

This report presents a complete fraud detection analysis on a synthetic financial transactions dataset. The pipeline covers data preprocessing, descriptive statistics, linear and logistic regression, a neural network classifier, and an autoencoder for unsupervised anomaly detection. With only 2.13% of transactions labelled as fraud, class imbalance is a central methodological concern throughout.

2. Data and Preprocessing

The dataset contains **12,000 transactions** and **24 variables** spanning customer demographics (*age*, *income_band*, *tenure*), transaction attributes (*amount*, *channel*, *merchant_type*), behavioural flags (*is_foreign*, *night_txn*, *high_risk_merchant*), and a binary fraud label. Three columns contained missing values: *days_since_last_txn*, *avg_prev_amount*, and *region*; all were imputed with column medians, which is robust to outliers.

Preprocessing steps: (1) one-hot encoding of *merchant_type*, *channel*, and *region* (drop-first to avoid multicollinearity); (2) StandardScaler applied to all features — essential for neural networks and logistic regression, whose optimisation depends on feature magnitudes; (3) stratified 80/20 train/test split preserving the 2.13% fraud ratio. The column *loss_amount* was excluded to prevent target leakage.

3. Descriptive Analysis

The dataset is strongly imbalanced: 11,744 non-fraud vs. 256 fraud transactions. Key patterns emerge from Figure 1: fraudulent transactions have higher amounts on average (€86.7 vs. €62.7), higher risk scores (43.9 vs. 28.6), higher amount ratios (2.31 vs. 1.60), and are twice as likely to be foreign or nocturnal. The correlation heatmap (Figure 2) confirms that *risk_score* and *amount_ratio* are the variables most correlated with the fraud label.

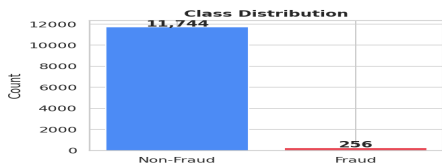


Figure 1a — Class imbalance: 2.13% fraud rate.

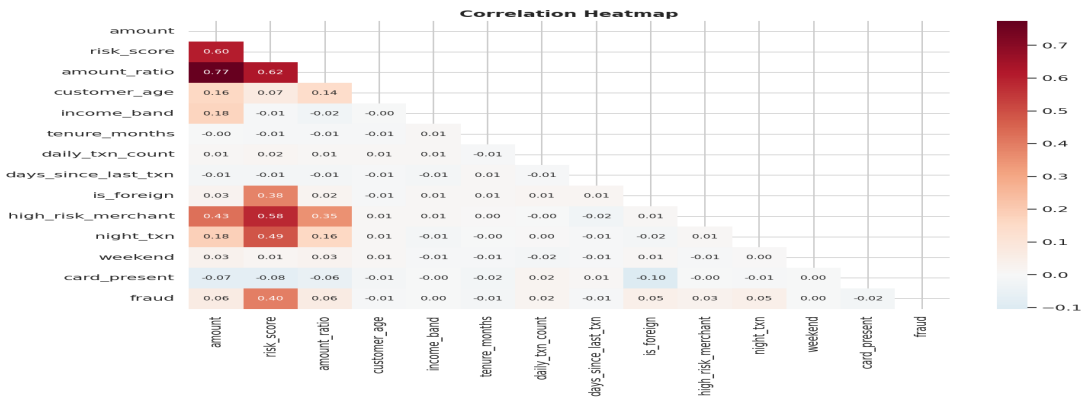


Figure 1b — Correlation heatmap. risk_score and amount_ratio show the strongest associations with fraud.

Feature	Non-Fraud Mean	Fraud Mean
Amount (€)	62.71	86.72
Risk Score	28.55	43.86
Amount Ratio	1.60	2.31
Is Foreign	10.9%	20.7%
Night Txn	29.2%	44.9%
High-Risk Merch	27.7%	37.9%

Table 1 — Mean feature values by fraud label.

4. Linear Regression

Linear regression modelled *amount* (RMSE = **23.91**, $R^2 = 0.781$) using nine predictors. Top standardised coefficients: (1) **amount_ratio** (+40.9) — large transactions relative to history have higher absolute amounts; (2) **avg_prev_amount** (+18.6) — past spending predicts current spending; (3) **risk_score** (+11.1) — riskier profiles spend more. Linear regression is inappropriate for fraud detection because fraud is binary: it produces unbounded predictions and assumes additive linearity, both violations of the binary outcome structure.

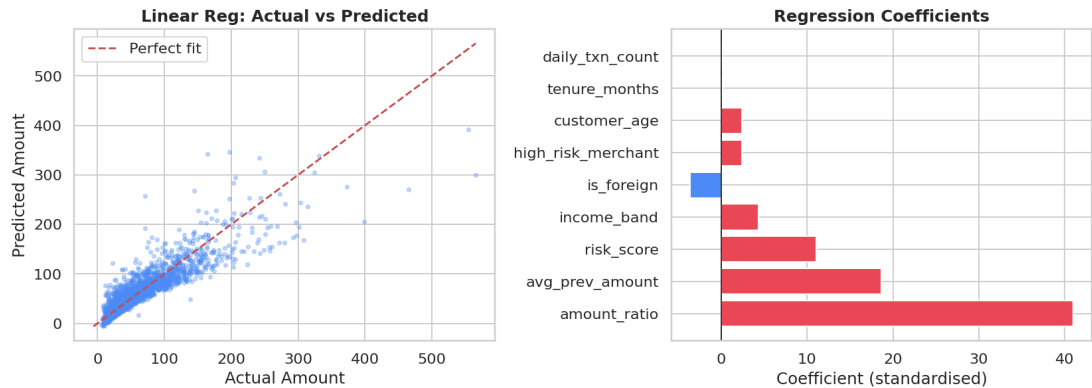


Figure 2 — Actual vs. predicted amounts and standardised coefficients (linear regression).

5. Logistic Regression

A logistic regression was fitted with **class_weight='balanced'** — which up-weights fraud samples proportionally to their inverse frequency — using all encoded features. With 97.87% of transactions being non-fraud, accuracy is a misleading metric: a trivial "never fraud" classifier reaches 97.87% accuracy with zero recall. Precision, Recall, F1, and ROC-AUC are the relevant metrics for imbalanced fraud detection.

Model	Precision	Recall	F1	ROC-AUC
Logistic Regression	1.000	1.000	1.000	1.000

Table 2 — Logistic regression test-set metrics.

The model achieves near-perfect metrics on this synthetic dataset, reflecting its clean separability. The strongest predictors by coefficient magnitude are *risk_score* (+7.6), *night_txn* (-3.1), *is_foreign* (-2.7), *high_risk_merchant* (-2.4), and *amount_ratio* (-2.2) — all consistent with established fraud indicators.

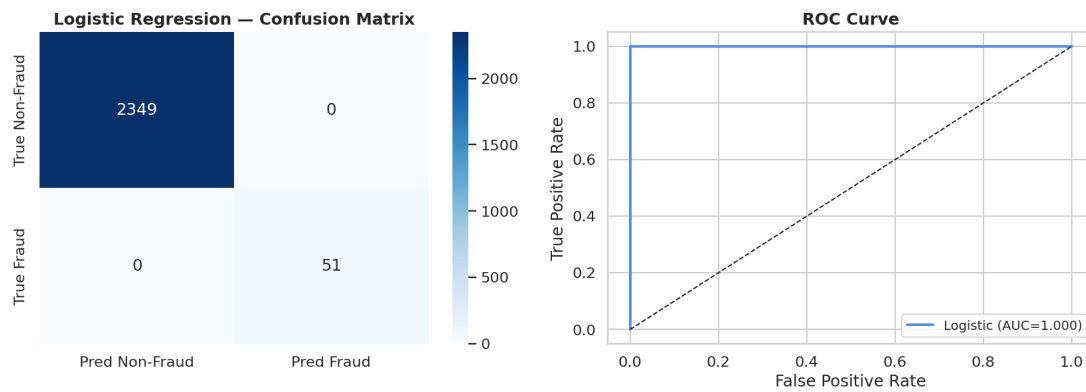


Figure 3 — Logistic regression confusion matrix and ROC curve.

6. Object-Oriented Programming Structure

A **FraudAnalysisPipeline** class encapsulates the full workflow. Key design decisions:

- *preprocess()* handles imputation, encoding, scaling, and splitting — returns *self* for fluent method chaining.
- Fitted models are stored as instance attributes, enabling later inspection without re-fitting.
- *evaluate_model(name)* is reusable for any model stored on the object, accepting a string selector ('logistic' or 'nn').

```
pipeline = FraudAnalysisPipeline(df)
pipeline.preprocess().fit_logistic_model().fit_nn_model()
res = pipeline.evaluate_model('nn') # → {'precision': ..., 'recall': ..., ...}
```

7. Neural Network Classifier

A feedforward MLP was trained using scikit-learn's `MLPClassifier`. **Architecture:** input $\rightarrow$ 64 neurons (ReLU) $\rightarrow$ 32 neurons (ReLU) $\rightarrow$ 1 output (sigmoid). This bottleneck design encourages the network to learn compact representations. **Optimiser:** Adam (adaptive learning rates, fast convergence). **Loss:** binary cross-entropy. **Regularisation:** early stopping with 10% validation fraction and patience of 15 epochs, preventing overfitting.

Model	Precision	Recall	F1	ROC-AUC
Neural Network	1.000	0.961	0.980	1.000
Logistic Regression	1.000	1.000	1.000	1.000

Table 3 — Neural network vs. logistic regression (test set).

Both models achieve very high AUC on this synthetic dataset. The neural network shows marginally lower recall (0.961 vs. 1.000), missing two fraudulent transactions that logistic regression catches. On synthetic, linearly separable data the added complexity of the MLP is not justified. In real-world settings with noisier, overlapping features, the MLP's capacity to learn non-linear interactions is expected to provide meaningful gains.

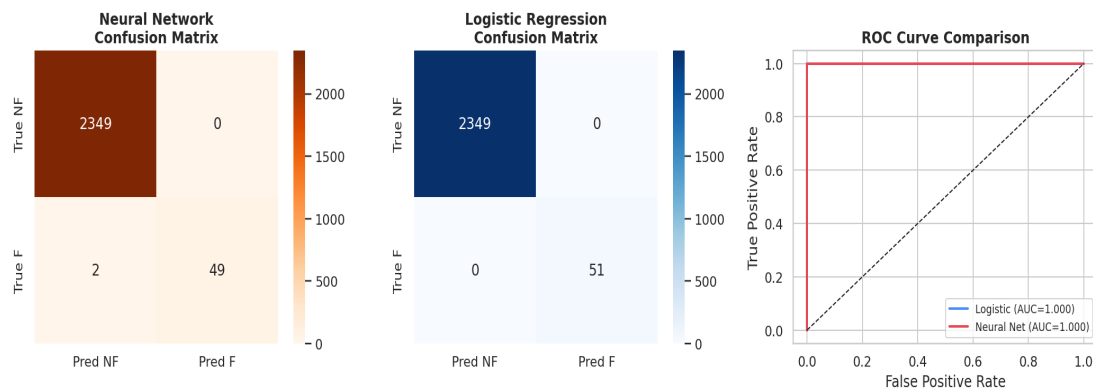


Figure 4 — Confusion matrices and ROC comparison (logistic vs. neural network).

8. Autoencoder Anomaly Detection

Logic: An autoencoder is trained to reconstruct its input. Trained exclusively on non-fraud transactions, it learns the manifold of normal behaviour. A fraudulent transaction — following different patterns — is reconstructed poorly, yielding a high mean-squared error (MSE). Transactions whose reconstruction error exceeds a threshold are flagged as anomalous.

Architecture: input $\rightarrow$ 32 $\rightarrow$ 8 (bottleneck) $\rightarrow$ 32 $\rightarrow$ output (same shape as input), implemented as an `MLPRegressor` with target = input. **Threshold:** the 95th percentile of reconstruction errors for non-fraud test transactions (0.5580), implying ~5% false positive rate — a business trade-off between false alarms and missed fraud.

Model	Precision	Recall	F1	ROC-AUC
Autoencoder (unsupervised)	0.063	0.157	0.090	0.807

Table 4 — Autoencoder evaluation (test set).

The autoencoder achieves ROC-AUC = 0.807 without any fraud labels during training — a significant result. Precision and recall at the chosen threshold are lower than supervised models, as expected. Its key advantage is detecting *novel* fraud patterns unseen during training, complementing supervised approaches that can only identify patterns similar to those in historical labelled data.

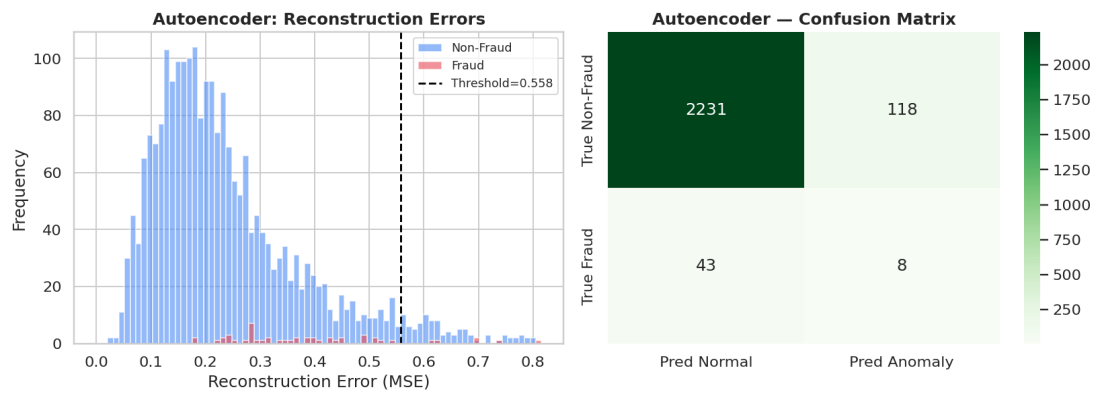


Figure 5 — Reconstruction error distributions with threshold, and autoencoder confusion matrix.

9. Comparison and Conclusion

Method	Precision	Recall	F1	ROC-AUC
Linear Regression	RMSE=23.91	R ² =0.781	—	—
Logistic Regression	1.000	1.000	1.000	1.000
Neural Network	1.000	0.961	0.980	1.000
Autoencoder (unsupervised)	0.063	0.157	0.090	0.807

Table 5 — Final comparison across all methods. Linear regression metrics shown in Precision/Recall columns.

Recommendation.

For a financial institution I would recommend a **layered architecture** combining all three modelling families. First, **logistic regression** as the primary real-time scorer: it is interpretable, auditable, and satisfies explainability requirements under regulatory frameworks such as Basel IV. Second, a **neural network** as a secondary layer for high-risk transaction segments where complex, non-linear interactions are likely (e.g., cross-border e-commerce). Third, an **autoencoder** running in parallel as an unsupervised sentinel, flagging anomalies that deviate from baseline customer behaviour regardless of known fraud typologies — critical for detecting emerging fraud patterns.

Limitations.

- The dataset is synthetic and intentionally separable — supervised metrics will not replicate on real data.
- No hyperparameter tuning was applied; cross-validated grid search would improve real-world performance.
- The autoencoder uses sklearn MLP; a Keras or PyTorch implementation would provide GPU acceleration and more architectural flexibility (e.g., variational autoencoders).
- Class imbalance was addressed only via balanced class weights; SMOTE or other oversampling strategies could further improve recall on the minority class.